

What is LLM confidence?

According to Ruth Miller, LLM confidence is “... (the) model’s internal measure of how certain it is about its output”. This measure is usually based on the tokens utilized by the model during the generation of the output.

Common methods used for calculating LLM confidence

Log probabilities/Logprobs summary: “Indicate the likelihood of each token occurring in the sequence given the context.” Calculated as $\log(p)$ where p is the probability of a token occurring at a specific position based on the previous tokens in the context. The likelihood of each token can be summed to determine the likelihood of a sequence. Logprobs are on a logarithmic scale, meaning that a slight difference in logprob values can cause a large difference between the probabilities calculated (Miller).

Token probabilities/Logits summary: Less complex, therefore should ideally be used for more simple tasks such as machine translation and short form Q&As.

Hidden-State Probing: Uses the model’s “internal state probe” during its response to a question as a signal used to estimate its confidence in the response. This signal shows whether the machine had some prior “knowledge” regarding the information it was asked about. This requires data that is labeled for correctness, which can make this method a more tedious process.

Self-Evaluation: Asking the model for a reasoning as to why its response was or wasn’t correct while calculating the probabilities of it outputting that it was true/false. It is a more straightforward method that requires less work/fine-tuning but is prone to faulty self-assessments, making it more useful in interactive settings such as code assistants or reasoning models.

Monte-Carlo/Ensembles: Sampling multiple responses that the model gives to slight variations of the same prompt and observing the variability. This method captures the certainty the model might have in a given fact that it is outputting consistently and allows for the detection of hallucinations that may appear in only some. It can be computationally costly but useful for more complex reasoning tasks.

Linguistic Cues (External method): Analyzing linguistic cues that point to uncertainty such as the use of “I think”, “certainly”, or “obviously” in its responses. Mostly useful for user-facing assistants, less complex such as customer support bots.

Self-Consistency (External method): Very similar to the Monte-Carlo method but instead of sampling the responses for calculating their probability it simply checks for equivalence for indication of its confidence. Is similarly computationally costly as Monte-Carlo but very effective in evaluating complex reasoning.

Surrogate/Proxy Models: Building a smaller model trained to mimic the original model's responses, allowing for confidence estimation without access to the LLM's internal reasoning. With this, more powerful internal methods (such as logit-based) can be used to calculate the smaller model's confidence. The surrogate can also be built as a classification model that analyzes the LLM's prompt and response.

AI Trust Score for AntiCopyPaster

For AntiCopyPaster, the confidence score should measure how trustworthy the full refactoring pipeline is, not just how confident the LLM is. Since AntiCopyPaster is a code-refactoring tool, the score should focus on whether the tool correctly detected a clone, generated a reasonable refactoring, checked that the refactoring was useful, compiled the result, and preserved behavior through testing.

For this reason, we propose calling the metric **AI Trust Score** instead of only “confidence score.”

Definition

The **AI Trust Score** is a weighted score that estimates how safe and reliable an AI-generated Extract Method refactoring is before the developer applies it.

Unlike basic LLM confidence, the AI Trust Score focuses on software-engineering validation signals from the AntiCopyPaster workflow.

Proposed Formula

AI Trust Score =

20% Clone Detection Confidence

- 20% Refactoring Agent Confidence
- 20% Refactoring Usefulness
- 20% Compilation Result
- 20% Test / Behavior Preservation

In equation form:

AI Trust Score =
0.20(Clone Detection Confidence)

- 0.20(Refactoring Agent Confidence)
- 0.20(Refactoring Usefulness)
- 0.20(Compilation Result)
- 0.20(Test / Behavior Preservation)

Each component should be scored from 0 to 100. The final AI Trust Score will also be between 0 and 100.

Why Equal Percentages?

Since this is an initial proposed version, we are using equal weights because each component represents a major stage in the AntiCopyPaster workflow. The score is not based on final experimental data yet, so equal weighting avoids overclaiming that one stage is mathematically more important than another.

Later, the weights can be adjusted based on experiments, developer feedback, or which stages are shown to best predict successful refactoring suggestions.

Component Breakdown

1. Clone Detection Confidence — 20%

This measures how confident the system is that the pasted code is actually a clone of existing code. If the clone detection is weak, then the rest of the refactoring suggestion is built on an unreliable foundation.

2. Refactoring Agent Confidence — 20%

This measures how strong or complete the Refactoring Agent's proposed Extract Method refactoring is. This could include whether the agent creates a clear helper method, identifies the correct duplicated logic, and proposes a reasonable replacement.

3. Refactoring Usefulness — 20%

This checks whether the suggested refactoring is actually meaningful. A useful Extract Method refactoring should remove duplication, preserve the original logic, and avoid producing an incomplete or unrelated change.

4. Compilation Result — 20%

This measures whether the refactored Java code still compiles. If compilation fails, the trust score should drop significantly or the suggestion should be blocked from being shown as safe.

5. Test / Behavior Preservation — 20%

This checks whether tests pass and whether the refactoring preserves the original program behavior. Since refactoring is supposed to improve code structure without changing functionality, this is an important part of trust.

Example Calculation

Suppose the system produces these component scores:

Clone Detection Confidence = 95

Refactoring Agent Confidence = 90

Refactoring Usefulness = 90

Compilation Result = 100

Test / Behavior Preservation = 100

AI Trust Score =

$0.20(95)$

- $0.20(90)$
- $0.20(90)$
- $0.20(100)$
- $0.20(100)$

AI Trust Score =

$19 + 18 + 18 + 20 + 20$

AI Trust Score = 95%

So the UI could display:

AI Trust Score: 95%

Safety Level: High

Clone Detection: High

Refactoring Proposal: Strong

Useful Extract Method: Passed

Compilation: Passed

Behavior Preserved: Yes

Note on LLM Confidence and Diff Safety

LLM confidence and diff safety can still be useful, but they should not be part of the main AI Trust Score formula right now. LLM confidence can be shown as optional metadata because model confidence does not always guarantee code correctness. Diff safety can also be shown as a supporting UI signal because a small, focused diff helps developers review the change more easily.

Example supporting details:

LLM Confidence: 90%

Diff Review: Small focused change

Important Note on Failure Cases

If a major pipeline step fails, the AI Trust Score should not be shown as a normal high score. Instead, the UI should show a blocked or unavailable state.

Example:

AI Trust Score: Unavailable

Reason: Model failed to initialize

Details: TensorFlow / Code2Vec native library issue

This is important because the tool should not silently fail. If the model, compiler, tests, or refactoring pipeline fails, the user should see a diagnostic message explaining what happened. This would make the UI more transparent and would support the AI Trust / Explainability section of the design.

Final Takeaway

For AntiCopyPaster, the AI Trust Score should represent developer trust in the full refactoring pipeline. Instead of relying only on LLM confidence, the score should combine the key stages of the workflow: clone detection, refactoring agent output, usefulness validation, compilation, and test/behavior preservation.

Literature:

1. <https://medium.com/@georgekar91/measuring-confidence-in-llm-responses-e7df525c283f>
2. <https://medium.com/@vatvenger/confidence-unlocked-a-method-to-measure-certainty-in-llm-outputs-1d921a4ca43c>
3. <https://arxiv.org/html/2406.03441v1>
4. <https://docs.together.ai/docs/inference/chat/logprobs>

Summary of methods (1. Karapetyan):

Methods for Measuring Confidence in Large-Language-Model Responses			
METHOD	WHAT IT DOES	PROS	CONS
Logit-Based Confidence	Uses token-level probabilities (logits) to compute likelihood of the generated sequence.	No extra training; works for any task; simple thresholding; easy ensemble extension.	Probability $\neq$ correctness; long outputs penalised; APIs may hide logits; over-confidence on fluent hallucinations.
Hidden-State Probing	Trains a lightweight classifier on internal activations to predict answer correctness.	Can tap into latent "knowledge" signals; catches subtle uncertainty; adaptable to reasoning checkpoints.	Requires labelled dataset & model access; may overfit cues; maintenance overhead when model/domain changes.
Self-Evaluation	Prompts the model to grade its own answer or state confidence verbally/numerically.	Intuitive; no extra models; can provide rationale; works on any API by prompt engineering.	Calibration quality depends on model; may echo over-confidence; doubles token cost; needs prompt craft.
Monte-Carlo / Ensembles	Samples multiple generations (or multiple models) and measures agreement/entropy.	Captures decision uncertainty; powerful for reasoning; model-agnostic; boosts accuracy via majority vote.	High compute & latency; correlated errors can mislead; aggregation rules may be ad-hoc.
Linguistic Cues	Parses hedging or bold language (or forced confidence tokens) in the answer text.	Human-readable; trivial to implement; improves UX transparency.	Stylistic bias; not correlated with truth unless calibrated; inconsistency across prompts.
Self-Consistency	Re-asks the question with variations; higher answer agreement $\Rightarrow$ higher confidence.	Excellent for chain-of-thought and hallucination detection; no extra training.	Adds cost/latency; consistent wrong answers fool metric; needs semantic comparison.
Surrogate / Proxy Models	Secondary model or classifier estimates confidence from Q-A pair.	Independent check; can mix many features; lightweight & cheap to run.	Proxy mismatch risk; requires training data; upkeep when base model evolves.
External Verification	Checks answer against trusted tools, databases, the web, or human reviewers.	Highest reliability when sources exist; grounds answers; enables citations & audits.	Latency & cost; requires pipelines/tooling; not viable for subjective/novel queries.

Highlighted:

Token Probabilities (1. karapetyan)

Log probabilities/logprobs (2. Miller) (4.)

Python package that calculates LLM output confidence based on log probabilities:

<https://github.com/VATBox/llm-confidence>

Together AI Documentation — Log probabilities

<https://docs.together.ai/docs/inference/chat/logprobs>

Tian et al. — “Just Ask for Calibration”

<https://arxiv.org/abs/2305.14975>

Zhang et al. — “Calibrating the Confidence of Large Language Models by Eliciting Fidelity”

<https://arxiv.org/abs/2404.02655>

AlOmar et al. — “On Preserving the Behavior in Software Refactoring”

<https://arxiv.org/abs/2106.13900>